



WeCloudData

Essential Functionalities

Pandas





Making Messy Data Useful

Pandas

Imagine you receive student records from different departments.

Department A:

Student ID	Marks
S0	85
S1	90
S2	75

Department B:

Student ID	Marks
S2	95
S3	88
S4	92



Making Messy Data Useful

Pandas

How do we:

- Match the correct students?
- Remove unwanted data?
- Find top performers?
- Calculate statistics?



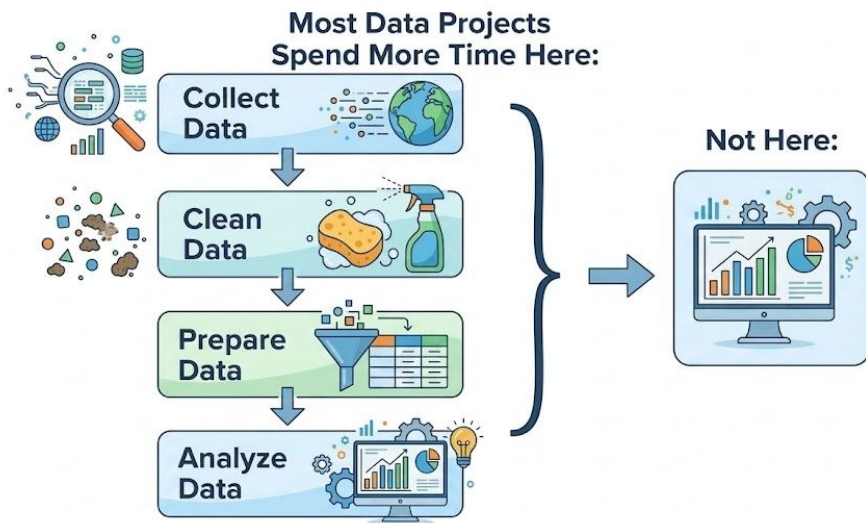
Real-world data is rarely clean, complete, or perfectly aligned.



Why Learn These Pandas Functionalities?

Pandas

Before Analysis Comes Preparation





WeCloudData

Reindexing



Reindexing

Pandas

Reindexing asks Pandas to return data in a requested label order. Labels that do not exist become missing values.

Current index	Marks
S1	85
S2	90
S3	75

Requested index
S1
S2
S3
S4

Result index	Marks
S1	85
S2	90
S3	75
S4	NaN

Existing labels are matched. New labels appear with NaN.



Why use Reindexing?

Pandas

- Put rows into a required order
- Add expected labels that are missing from the data
- Compare datasets with the same index structure
- Make missing records visible instead of silently ignoring them

```
df.reindex(["S3", "S1", "S4"])
```

Index	Marks
S3	75
S1	85
S4	NaN

Reindexing is about requested labels, not just filtering existing rows.



Reindex vs Selection

Pandas

Selection

“Which existing rows do I want?”

```
df.loc[["S1", "S2"]]
```

Index	Marks
S1	85
S2	90

Reindexing

“What labels/order should result have?”

```
df.reindex(["S1", "S2", "S4"])
```

Index	Marks
S1	85
S2	90
S4	NaN



WeCloudData

Dropping Entries



Dropping Rows and Columns

Pandas

Dropping is:

Removing unwanted data.

Drop a row

```
df.drop(index=["S2"])
```

Index	Marks
S1	85
S3	75

Drop a column

```
df.drop(columns=["Notes"])
```

Name	Age
Sara	22
Ahmed	21



Importance

When Would We Remove Entries?

When do we use “drop”:

- Remove duplicates
- Remove temporary columns
- Remove incorrect records

Example:

A dataset contains:

- Name
- Age
- Address
- Notes

Notes column is empty.

Drop it.





WeCloudData

Indexing, selection and Filtering



Select Data by Label or by Position

Pandas

Index	Name	Marks
S1	Sara	85
S2	Ahmed	90
S3	Lina	75

Use labels with `.loc[]`

```
df.loc["S1", "Marks"]  
# 85
```

Use integer positions with
`.iloc[]`

```
df.iloc[0, 2]  
# 85
```



Filtering Data Like a Detective

Pandas

Question

Which students scored above 80?

Example

Dataset:

Index	Name	Marks
S1	Sara	85
S2	Ahmed	90
S3	Lina	75

```
df[df["Marks"] > 80]
```

Marks



Id

S1	85
S2	90



Filtering Keeps Rows That Match a Condition

Filtering

```
df[df["Marks"] > 80]
```

Index	Name	Marks
S1	Sara	85
S2	Ahmed	90

- Filtering does not choose by row number.
- Filtering does not automatically sort the result.
- It simply keeps rows where the condition is True.

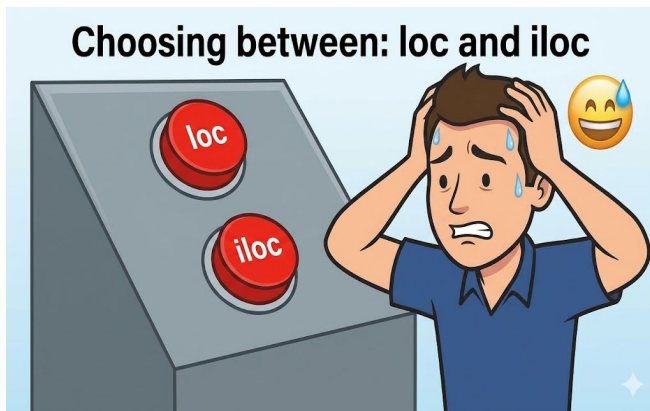
Question it answers: “Which rows satisfy this rule?”



Selecting Data with loc and iloc

Pandas

Task	Use	Example
Select by row/column label	<code>.loc[]</code>	<code>df.loc["S1", "Marks"]</code>
Select by numeric position	<code>.iloc[]</code>	<code>df.iloc[0, 1]</code>
Keep rows that meet a condition	Boolean filter	<code>df[df["Marks"] > 80]</code>





WeCloudData

Arithmetic and Data Alignment



Pandas Aligns Arithmetic by Labels

Pandas

Imagine having 2 series and we want to sum them together:

```
s1 = pd.Series([10, 20],  
               index=["S1", "S2"])
```

Label	s1
S1	10
S2	20

```
s2 = pd.Series([5, 8],  
               index=["S2", "S3"])
```

Label	s2
S2	5
S3	8



$s1 + s2$

Label	Result
S1	NaN
S2	25
S3	NaN

Pandas aligns by labels, not by position.



Why Alignment Matters

Pandas

In our previous example

Position-based thinking

$10 + 5 = 15?$

This may combine different students.

Label-based alignment

S2: $20 + 5 = 25$

Only matching labels are combined.

Alignment **protects the meaning of the data** when two objects are not in the same order or do not contain the same labels.



Alignment Is Helpful, But Check the Labels

Pandas

- Automatic alignment follows labels exactly
- Unmatched labels create NaN
- Misspelled or inconsistent labels can produce unexpected results
- Duplicate labels can make the result harder to interpret

```
# "S1" and "s1" are  
different labels  
series_a + series_b
```

**Good habit: inspect the index
before combining data.**





WeCloudData

Function Application and Mappings



Apply Function

Pandas

Suppose we want to create a new column to indicate whether students pass the exam, with **apply function**, we can easily do so:

```
def pass_fail(mark):  
    return "Pass" if mark >= 60 else  
        "Fail"  
  
df["Result"] =  
df["Marks"].apply(pass_fail)
```

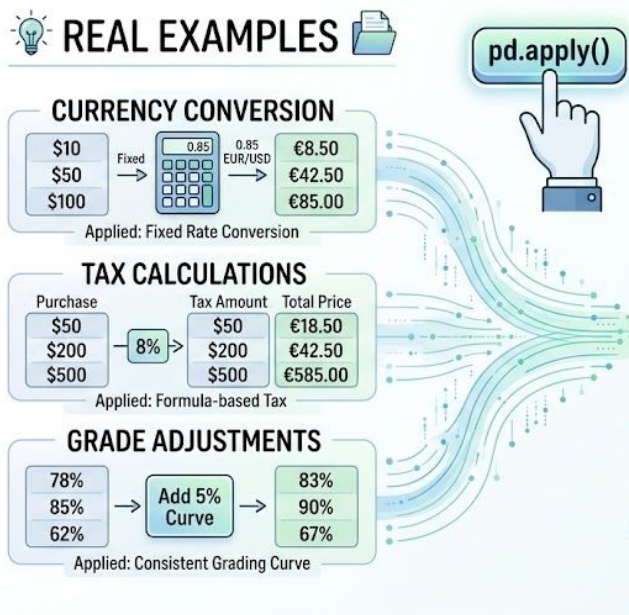
Index	Marks	Result
S1	85	Pass
S2	90	Pass
S3	55	Fail

apply = run a function for each value, row, or column



Apply Function Examples

Pandas



Apply = Repeat an operation automatically



Map Function

Pandas

Map is useful when each value should be **replaced or converted using a dictionary or function**.

```
grade_map = {"A": "Excellent", "B": "Good", "C":  
             "Needs Review"}  
df["Grade_Label"] = df["Grade"].map(grade_map)
```

Index	Grade	Grade_Label
S1	A	Excellent
S2	B	Good
S3	C	Needs Review



Apply vs Map Function

Pandas

	map	apply
Main use	Convert values in a Series	Run custom logic
Common input	Dictionary or function	Function
Works with	Series	Series or DataFrame
Use Cases Example	A → Excellent	mark → Pass/Fail

Simple rule

- map for value conversion
- apply for custom logic



WeCloudData

Sorting and Ranking



Sorting and Ranking

Pandas

Student	Marks
Sara	85
Ahmed	90
Lina	75

- Sorting Reorders Rows
- Ranking Assigns Positions

```
df.sort_values("Marks")
```

Student	Marks
Lina	75
Sara	85
Ahmed	90

Order

```
df["Rank"] = df["Marks"].rank(ascending=False)
```

Student	Marks	Rank
Sara	85	2
Ahmed	90	1
Lina	75	3

Ranking



Order

Pandas

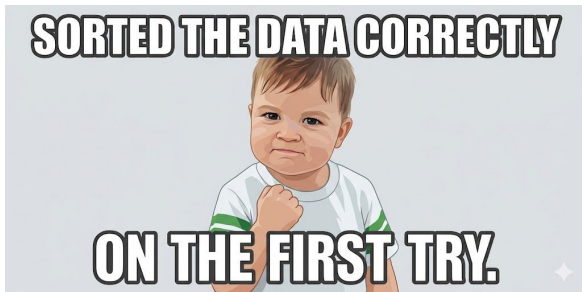
Sorting Answers:

“What Order Should the Rows Appear In?”

- Sort by one column, such as Marks
- Choose ascending or descending order
- Useful for finding top or bottom records
- **Sorting changes the row order in the result.**

```
df.sort_values("Marks", ascending=False)
```

Student	Marks
Ahmed	90
Sara	85
Lina	75





Rank

Pandas

Ranking Answers:

“What Position Does Each Value Get?”

- Ranking creates a rank value for each row.
- Ties need a ranking rule, such as average, min, or dense.
- **The original row order does not have to change.**

```
df["Rank"] =  
df["Marks"].rank(ascending=False)
```

Student	Marks	Rank
Sara	85	2
Ahmed	90	1
Lina	75	3



WeCloudData

Axis Indexes with Duplicate Values



Duplicate Indexing

Pandas

Imagine a school database:

Student ID	Name	Marks
S1	Abdullah	80
S1	Ahmed	90
S2	Sara	95

Student ID S1 appears twice!



Duplicate Indexing Example

Pandas

A duplicate index means the same index label appears more than once.

Index	Name	Marks
S1	Abdullah	80
S1	Ahmed	90
S2	Sara	95

The two S1 rows are different records.

The index label is duplicated.

This may be valid, but selections can return multiple rows.

```
df.loc["S1"]  
# returns both S1 rows
```



When Duplicate Indexes Can Be Useful or Risky

Pandas

Can be useful

- Multiple attempts by one student
- Multiple events for one device
- Multiple transactions for one customer

Can be risky

- Selecting one label returns many rows
- Alignment may be harder to inspect
- Reindexing may become ambiguous



How to Check for Duplicate Indexes

Pandas

- Use **is_unique** for a quick **yes/no** check.
- Use **duplicated()** to see **which labels** repeat.
- **Decide whether the duplicate index is expected** before changing it.

```
df.index.is_unique  
# False
```

```
df.index.duplicated()  
# array([False,  True,  False])
```



Summary

Pandas

Functionality	Purpose
Reindex	Match/reorder labels
Drop	Remove rows/columns
loc	Select by label
iloc	Select by position
Alignment	Match labels automatically
apply	Run custom logic
map	Replace values
Sort	Order records
Rank	Assign positions



Summary

Pandas

Use These as a Pandas Cleaning and Preparation Toolkit

Understand labels
Index

Match structure
Reindex + Alignment

Choose data
loc / iloc / Filter

Clean columns/rows
Drop

Transform values
Apply + Map

Organize results
Sort + Rank

These tools are **not a fixed pipeline**. Choose the one that matches the data problem in front of you.